A Deterministic Architecture for Long-Horizon AI Systems

Taylor Maddry

Austin, Texas
January 2026

Abstract

AI systems intended to operate as long-running autonomous or adaptive systems face a class of failure not captured by traditional notions of error, robustness, or performance degradation. Even when locally correct and internally consistent, such systems may become unsafe to rely upon as accumulated state and derived artifacts gradually acquire de facto authority. This failure arises not from adversarial behavior or misalignment, but from implicit authority drift driven by reuse, dependency, and ungoverned promotion of information over repeated autonomous decision cycles.

We present a deterministic architecture for long-horizon autonomous and adaptive systems that prevents implicit authority drift by construction. The architecture enforces a strict separation between immutable substrate law and governed mutable state, treats authority as an explicit, versioned system property, and constrains all authority-bearing change to discrete, inspectable, and reversible governance actions. A deterministic execution pipeline with mandatory refusal semantics ensures that autonomous action selection and state transition occur only when substrate-enforced authority conditions are satisfied.

The system employs a three-tier memory architecture that distinguishes ephemeral working context, persistent non-authoritative artifacts, and authoritative epistemic basis. Research, analysis, and generative components may be used freely within autonomous and adaptive workflows, but are treated as epistemically inert by default, producing artifacts rather than authority. Provenance captures not only dependency relationships but epistemic admissibility, enabling deterministic replay and post hoc validation of autonomous decisions under fixed authority configurations.

This work does not aim to produce correct answers, aligned behavior, or adaptive intelligence. Instead, it presents a deterministic architectural framework that constrains how long-running autonomous systems may evolve, accumulate state, and exercise authority over time. By enforcing explicit governance, reversibility, and determinism under accumulation, the architecture supports autonomous operation in high-cost-of-failure domains where auditability, bounded authority, and correction without implicit drift are required.

Table of Contents

1. Introduction....1
2. Problem Statement and Design Requirements....1
3. Conceptual Framing: A Formal System, Not an Agent....2
4. Core Architectural Separation: Substrate and Strata....3
5. Deterministic Pipeline and Refusal Semantics....4
6. Memory Architecture and Epistemic Roles....5
7. Research Without Authority....6
8. Governance, Promotion, and Correctable Authority....7
9. Authority Scope and Roles....7
10. Provenance, Determinism, and Replay Across Authority Configurations....8
11. Limitations, Non-Goals, and Scope....9
12. Discussion and Conclusion....10
Appendix A: Definitions....12
Appendix B: Illustrative Failure Mode Under Accumulation....14
Appendix C: An Abstract Model of Authority Drift....15
References....18

1. Introduction

Long-horizon autonomous AI systems inevitably accumulate state and derived artifacts across decisions, creating dependencies that traditional notions of local correctness and robustness are not designed to constrain. These systems may remain locally correct at every step and exhibit internal consistency, while nevertheless becoming unsafe to rely on for decisions that depend on explicit authority and verifiable grounding. The failure emerges gradually from normal operation rather than from sudden faults or adversarial conditions. This failure follows from the reuse of accumulated state across decisions, a condition that long-horizon systems cannot avoid.

As systems accumulate state and artifacts that increasingly depend on one another, implicit authority drift changes what they treat as authoritative without a discrete moment of authorization. Advisory outputs are reused, summarized, and depended upon until they functionally ground decisions. Provenance chains grow longer and more coherent, yet increasingly detached from any explicit epistemic basis. Authority is not assigned or reviewed; it is inferred from dependency. Over time, the system's behavior appears stable while its justification becomes irreproducible.

This failure mode is difficult to detect precisely because no single action is incorrect. Each step is locally rational, often improving efficiency or usability. Logging, versioning, and audit trails may remain intact, but they record only how decisions followed from prior artifacts, not whether those artifacts were ever granted epistemic authority. By the time the problem becomes visible, there is no clear boundary to inspect, no discrete change to reverse, and no principled way to restore a known-good state. The system has not violated its rules; its rules were insufficient to prevent authority from shifting implicitly.

Existing AI architectures and safety mechanisms assume that authority is either static or externally enforced. They focus on controlling behavior and improving outputs, with transparency treated as a supporting concern rather than a structural constraint. These measures are effective against many classes of failure, but they do not address the long-horizon problem of authority drift driven by accumulated dependency. In the absence of structural constraints that distinguish advisory context from epistemic basis and require explicit governance for authority transitions, reuse and accumulation alone are sufficient to undermine correctness over time.

This paper argues that preventing implicit authority drift under accumulation requires treating authority as a first-class, versioned concept enforced by immutable system law. Long-horizon AI systems must separate unchanging enforcement mechanisms from governed, inspectable state; must make authority transitions discrete, reviewable, and reversible; and must ensure that provenance and replay capture not only dependency but epistemic status. We present an architecture that satisfies these requirements by construction, enabling bounded, auditable evolution without implicit learning or ungoverned change. Key terms are used in a precise sense in this work and are defined in Appendix A.

2. Problem Statement and Design Requirements

The failure described in the preceding section arises from a structural absence: the lack of enforced semantics for authority in systems that accumulate and reuse state. Any system that permits derived artifacts to influence subsequent decisions without an explicit, governed transition of epistemic status is vulnerable to implicit authority drift, regardless of scale, domain, or intended lifespan.

We therefore frame the problem independently of time. The relevant condition is accumulation. Whenever a system allows information produced in one decision context to be reused in another, it must answer a foundational question:

Under what conditions does reused information gain the right to ground future decisions?

In most contemporary systems, this question is either answered informally or not answered at all. Authority is inferred from proximity, convenience, or dependency rather than assigned through explicit, inspectable mechanisms.

From this framing, a set of non-negotiable design requirements follows. These requirements define the minimum conditions under which correctness can be preserved under accumulation in systems that reuse state across decisions.

R1. Explicit Authority Semantics. The system must represent epistemic authority as a first-class concept. It must be possible to distinguish, structurally and enforceably, between advisory context and authoritative basis. Labels, conventions, or documentation are insufficient if they do not affect what the system is permitted to use to justify decisions or actions.

R2. Governed Authority Transitions. Any change in what the system treats as authoritative must occur through an explicit, discrete, and reviewable process. Gradual or implicit promotion through reuse, summarization, or dependency is prohibited. Authority transitions must be inspectable and attributable.

R3. Immutable Enforcement of Authority Rules. The rules that govern authority semantics and transitions must themselves be immutable at runtime. A system that can reinterpret or weaken its own authority constraints through accumulated state recreates the original failure mode these constraints are meant to prevent.

R4. Separation of Enforcement and State. Mechanisms that enforce authority and governance must be strictly separated from the state that evolves under those mechanisms. Enforcement logic must not be modifiable by the same processes it constrains.

R5. Provenance with Epistemic Meaning. Provenance records must capture not only dependency relationships but epistemic status. It must be possible to determine, for any decision, which inputs were authoritative, which were advisory, and under which rules that distinction was applied. Provenance that records dependency without authority semantics is insufficient [8].

R6. Deterministic Replay and Reversibility. The system must support deterministic replay of decisions under a fixed authority configuration, and it must be possible to reason about reverting to a prior authoritative state. Without replay and reversibility, authority transitions cannot be validated or safely corrected [7].

These requirements rule out many existing architectures that permit accumulation without enforced authority semantics. Systems that rely on informal conventions, emergent trust, statistical confidence, or gradual adaptation cannot satisfy them without fundamental redesign. The remainder of this paper presents an architecture that meets these requirements by construction, providing a foundation for AI systems that remain correct, inspectable, and governable under unbounded accumulation.

3. Conceptual Framing: A Formal System, Not an Agent

Satisfying these requirements requires abandoning agent-centric interpretations and instead treating the system as a formal construct governed by explicit, enforceable rules. The requirements outlined in the previous section deliberately avoid assumptions about cognition or intention, focusing instead on enforceable system properties rather than agent-centric interpretations of autonomy. Many failures attributed to long-running AI systems are framed in terms borrowed from human or agent-based metaphors: belief, memory, intention, trust, or learning.

Anthropomorphic language obscures the underlying structure of the problem by shifting attention away from enforceable system properties and toward interpretive narratives.

We therefore frame the system under study as a formal system governed by explicit rules, state, and transitions. In this framing, the system is not an agent with beliefs or goals. It is a deterministic machine that accepts inputs, produces outputs, and mutates state subject to invariant constraints.

Correctness is defined by whether every state transition is permitted by the system's governing rules and whether those rules themselves remain stable [4].

The source of authority drift is structural rather than a consequence of agent cognition or internal modeling. It arises when the formal structure of a system allows information produced in one context to influence another without an explicit rule authorizing that influence. Anthropomorphic language mislocates the failure by suggesting problems of judgment or interpretation where the real issue is the absence of enforceable constraints [1].

By treating the system as a formal construct, we can state its obligations precisely. State is data, not memory, and outputs are artifacts, not beliefs. Reuse is treated as a transformation rather than as a source of epistemic learning, and authority is a property assigned by rule rather than an emergent trait inferred from behavior. These distinctions are necessary to reason about correctness under accumulation [6].

This framing also clarifies the role of probabilistic or generative components. Such components may be used to produce candidate artifacts, summaries, or plans, but they do not participate in authority assignment or enforcement. From the perspective of the formal system, their outputs are inputs like any other, subject to the same authority semantics and governance rules. Correctness depends on the authority semantics applied to component outputs, not on whether those outputs are produced deterministically or probabilistically.

With this framing in place, we can now introduce an architecture that enforces the requirements of Section 2 as invariant properties of a formal system. The next section describes a separation between immutable enforcement machinery and governed mutable state that makes implicit authority drift structurally impossible.

4. Core Architectural Separation: Substrate and Strata

To satisfy the requirements of Section 2 within the formal framing established in Section 3, the system must make a foundational architectural distinction between what is permitted to change and what is not. We call these two domains Substrate and Strata. This separation provides the architectural basis for enforcing authority semantics under accumulation, rather than relying on informal or procedural controls.

The Substrate comprises all immutable enforcement machinery of the system. It defines the rules under which state may be interpreted, transformed, and mutated. These rules include authority semantics, governance constraints, validation logic, and the conditions under which state transitions are permitted. The Substrate is version-static at runtime: once defined, it cannot be altered by any process operating within the system. It does not learn, adapt, or evolve. Its sole responsibility is to enforce invariants deterministically.

The Strata comprises all mutable state that exists under the governance of the Substrate. This includes operational data, accumulated artifacts, configuration values, and any structures that may change over time as the system is used. State within the Strata may grow, shrink, or be revised, but only through transitions validated by the Substrate. The Strata contains no logic that can reinterpret or weaken the rules that govern it [5].

Within this separation, authority is treated as a first-class system property whose representation and transition are enforced by immutable system law, and are therefore governed by the Substrate rather than encoded within the Strata. Authority arises from explicit state transitions governed by Substrate rules that are not themselves subject to runtime modification. This distinction is foundational: systems that encode authority semantics within mutable Strata structures, policy, process, or interpretation operate in a different design space, even when workflows or governance intent appear similar. This work examines the structural consequences of locating authority semantics in the Substrate rather than in the mechanisms that act upon the Strata [10].

This separation directly enforces several of the requirements identified earlier. Because authority semantics live exclusively in the Substrate, they cannot be modified through reuse or accumulation within the Strata (R3). Because the Substrate validates every mutation to the Strata, authority transitions can be made explicit, discrete, and inspectable (R2). Because the Strata contains all evolving artifacts while the Substrate remains fixed, enforcement and state are structurally separated (R4).

The Substrate-Strata distinction also clarifies what constitutes a meaningful system change. Changes to the Strata represent evolution within a fixed set of rules. Changes to the Substrate represent a change to the rules themselves and therefore constitute a new system version rather than an internal adaptation. This distinction prevents gradual reinterpretation of authority semantics and makes system evolution explicit rather than implicit.

Accumulation within the Strata is expected and permitted; it is the normal operation of the system. What is prohibited is accumulation that alters the rules by which authority is assigned or exercised. By fixing those rules in the Substrate and confining all growth to the Strata, the system ensures that accumulation increases context without increasing authority.

In the following sections, we describe how this separation is realized concretely under the constraints defined in Section 2.

5. Deterministic Pipeline and Refusal Semantics

The Substrate-Strata separation establishes what may change and what may not, but it does not yet explain how work is performed. To do so, the system executes all requests through a deterministic pipeline that transforms inputs into outputs while enforcing authority semantics at every stage. This pipeline exists to operationalize the system's authority and provenance constraints, not to optimize execution. The deterministic pipeline is the mechanism by which the Substrate enforces authority semantics with respect to the Strata during execution.

At a high level, each request is processed as a sequence of bounded stages: interpretation of the request, construction of candidate actions or artifacts, validation against authority and governance rules, and either commitment or refusal. Intermediate products of this process may be generated, inspected, and discarded, but only committed outputs are permitted to affect Strata. No stage may bypass validation, and no later stage may reinterpret the outcome of an earlier validation.

Refusal is treated as a correctness-preserving outcome of pipeline execution. If a request cannot be satisfied without violating authority semantics, governance constraints, or invariants enforced by the Substrate, the system must refuse to proceed. Refusal cannot be treated as an error condition, degraded performance, or exceptional state; it is a correctness-preserving result required to prevent unauthorized or unverifiable transitions.

This property is critical to preventing authority drift. In systems that lack explicit refusal semantics, pressure to produce an output often leads to gradual relaxation of constraints or informal workarounds. By contrast, a

deterministic pipeline with mandatory refusal ensures that unmet preconditions cannot be silently bypassed. A request either satisfies the rules under which authority is exercised, or it does not execute.

Determinism plays a complementary role. Given the same inputs, governed mutable state, and Substrate version, the pipeline must produce the same outcome, whether that outcome is commitment or refusal. This property enables replay and audit: it must be possible to re-execute a prior request under identical conditions and obtain the same result. Without determinism, provenance records lose explanatory power and authority transitions cannot be reliably inspected.

Determinism here does not preclude the use of probabilistic or generative components within the pipeline. Such components may contribute candidate artifacts or analyses, but their outputs are treated as advisory inputs subject to validation. Any nondeterminism they introduce must be captured, bounded, or rendered inert with respect to committed state so that replay semantics remain well defined.

By enforcing a deterministic pipeline with explicit refusal semantics, the system ensures that all state transitions are deliberate, inspectable, and governed. Authority cannot be exercised implicitly through partial execution, fallback behavior, or convenience shortcuts. Every committed change either satisfies the rules enforced by the Substrate or does not occur at all.

6. Memory Architecture and Epistemic Roles

To make authority enforcement and provenance meaningful across executions, the system must also constrain how state is persisted and reused. To enforce explicit authority semantics and provenance with epistemic meaning, the system classifies persisted and ephemeral state according to epistemic role and lifecycle constraints. We refer to these classifications as memory tiers, using the term strictly to denote storage and persistence behavior, not cognitive function. As established in Section 3, the system does not possess memory in an experiential or agentic sense; it maintains state under rules. The tiers described here are categories of state governed by different authority and persistence guarantees.

Tier A: Ephemeral Working State. Tier A comprises transient state created during the execution of a single request. This includes intermediate representations, candidate artifacts, planning structures, and validation context produced within the deterministic pipeline. Tier A state exists only for the duration of request processing and is discarded upon commitment or refusal. It is never persisted, never referenced across requests, and never eligible to ground future decisions. Tier A state is non-authoritative by construction and plays no role in provenance beyond supporting immediate computation.

Tier B: Non-Authoritative Persisted Artifacts. Tier B comprises persisted artifacts that may be accumulated, inspected, and reused but are explicitly non-authoritative. Examples include summaries, analyses, research notes, intermediate reports, and other derived materials produced by the system or provided as input. Tier B artifacts may inform future requests, but they cannot, by rule, ground decisions or justify state transitions. Their reuse does not confer authority, regardless of frequency, dependency, or internal consistency. Provenance records must reflect Tier B status explicitly so that dependency on these artifacts is never mistaken for epistemic grounding.

Tier C: Governed Authoritative State. Tier C comprises state that has been explicitly granted epistemic authority through a governed transition. This includes validated facts, approved configurations, committed policies, and any other information permitted to ground decisions or authorize actions. Promotion of information into Tier C is a discrete, reviewable event enforced by the Substrate in accordance with authority semantics and governance rules.

Tier C state is authoritative only within the scope defined by the authority configuration under which it was promoted.

These tiers collectively enforce Requirements R1 and R5. By separating advisory and authoritative roles structurally, the system prevents epistemic flattening and implicit promotion through reuse. By recording tier membership as part of provenance, the system preserves not only what was depended upon, but what was permitted to ground a decision at the time it was made.

Reversion of authoritative state operates at the level of Tier C authority configurations, not at the level of stored artifacts. Reverting to a prior authoritative state does not delete, collapse, or rewrite provenance. Artifacts and decisions produced under superseded authority configurations remain in the record, but they are no longer eligible to ground new decisions going forward. Reversion changes which portions of Tier C are active, not which portions of history exist.

This distinction ensures that provenance remains immutable and complete while authority remains contextual and versioned. The system can therefore support audit, replay, and correction without erasing evidence or obscuring how prior decisions were made. Accumulation of Tier B artifacts and historical Tier C states increases available context, but never increases authority without explicit governance [3].

With this memory architecture in place, the system can accumulate rich contextual state over time while maintaining strict control over epistemic authority. The next section builds on this foundation to describe how research, analysis, and external information can be incorporated without granting them authority by default.

7. Research Without Authority

The memory architecture described in the previous section enables the system to accumulate large amounts of contextual information without conflating accumulation with authority. This distinction is most critical in the context of research, analysis, and interaction with external information sources. In many AI systems, research outputs gradually acquire de facto authority through reuse, corroboration, or apparent confidence. The architecture described here explicitly prevents such escalation.

In this system, all research activity produces Tier B artifacts. These artifacts may include analyses, summaries of external sources, exploratory reasoning, comparative evaluations, and candidate explanations. They may be extensive, internally consistent, and repeatedly consulted. None of these properties confer epistemic authority. By construction, research outputs cannot ground decisions, authorize actions, or justify state transitions.

Research is treated as an input to human judgment, not as a source of truth. The system may assist by organizing information, highlighting relationships, or generating candidate interpretations, but it does not adjudicate correctness. Authority remains external to research and is exercised only through governed promotion into Tier C.

The separation between research and authority has two important consequences. First, the system remains correct even if all research artifacts are wrong, misleading, or contradictory. Errors in research do not propagate automatically into authoritative state. Second, disagreement and conflict among research artifacts are preserved rather than resolved implicitly. Contradictions among research artifacts are preserved for inspection rather than resolved through aggregation or confidence scoring.

When authoritative change is desired, research artifacts may be referenced as supporting material in a promotion proposal, but the proposal itself is a distinct object subject to governance. The system may assist in assembling such

proposals, but it does not promote research artifacts automatically or implicitly. Promotion requires explicit operator action and approval under the authority semantics enforced by the Substrate.

By enforcing a strict boundary between research and authority, the system avoids a common failure mode in which evidence gradually becomes policy through repetition or convenience. Accumulation of research increases available context and supports informed decision-making, but it does not alter what the system is permitted to treat as authoritative. This ensures that the system's epistemic basis remains stable, inspectable, and governed even as its informational context grows without bound.

8. Governance, Promotion, and Correctable Authority

The preceding sections establish that authority cannot emerge implicitly and that all authoritative state must be the result of explicit, governed promotion. What remains to be specified is how such governance operates in the presence of human fallibility. Any system intended to operate over long timespans must assume that operators will occasionally make incorrect judgments. The architecture described here is therefore designed not to prevent error, but to ensure that error is bounded, visible, and correctable.

Authority within the system is exercised exclusively through promotion into Tier C**.** Promotion is a discrete governance action that assigns epistemic authority to specific state under a defined scope and authority configuration. This action is always attributable to an operator and recorded as part of the system's provenance. The system itself does not autonomously grant authority; at most, it may assist by generating promotion proposals as Tier B artifacts. Such proposals have no authority and cannot affect system behavior unless explicitly approved.

Because promotions are explicit and versioned, incorrect promotions do not irreversibly compromise the system. If Tier B material is promoted in error, the resulting Tier C state becomes authoritative only under the authority configuration in which it was approved. Subsequent detection of error does not require reinterpretation of history or silent correction. Instead, the authority configuration may be reverted or superseded through a new governed transition, restoring a prior authoritative basis.

Correction does not entail erasure. Provenance records preserve the fact that the incorrect promotion occurred, the context in which it was approved, and the decisions that depended upon it. Reversion changes which authority configuration is active going forward, not which artifacts exist or which past decisions were made. This ensures that error remains inspectable and auditable rather than hidden or smoothed over.

This governance model establishes a clear asymmetry between error and correction. Errors are possible but localized. Correction is explicit and attributable, and does not rely on informal process or memory. Authority does not decay automatically, nor does it propagate implicitly. It changes only through governed action, and those actions remain visible indefinitely.

By treating incorrect authority as a governance error rather than a system failure, the architecture preserves long-horizon stability. The system remains usable and correct even under imperfect human judgment, provided that authority transitions remain explicit, inspectable, and reversible. This property is essential for any system intended to accumulate state indefinitely while remaining safe to rely upon for decisions that matter.

9. Authority Scope and Roles

The governance model described so far does not depend on a single authority-bearing actor; this section extends that model across scoped roles. The preceding sections describe authority as an explicit, governed concept exercised by

an operator. This terminology refers to a role rather than a specific individual or organizational structure. The architecture does not assume a single authority-bearing actor, nor does it depend on any particular distribution of responsibility. Instead, it defines constraints that any instantiation of authority must satisfy, regardless of how many operators exist or how they are organized.

Authority-bearing roles are defined by scope. Scope determines which classes of decisions an operator may authorize, under which conditions, and over which portions of system state. Scopes may overlap, but any such overlap must be explicit and enforced by the Substrate. No operator may exercise authority outside the scope assigned to them, and no action may derive authority from multiple scopes through aggregation or informal consensus unless such coordination is itself explicitly governed.

The system supports multiple operators by treating authority as non-compressible and non-transferable by default. Authority is not inferred from seniority, frequency of interaction, or historical reliance. Each governed action is attributed to a specific authority configuration that records which role exercised authority, under what scope, and at what time. Multi-operator configurations therefore increase deliberative capacity without altering the semantics of authority itself.

Coordination among operators, when required, is handled through explicit governance rules rather than implicit agreement. For example, certain authority transitions may require concurrence from multiple scoped roles, sequential approval, or explicit handoff. These coordination requirements are defined as part of the authority configuration and enforced by the Substrate in the same manner as any other invariant. There is no mechanism for authority to emerge from majority behavior, statistical agreement, or repeated reliance.

The architecture does not attempt to solve social or organizational governance. It provides a formal substrate in which such governance can be expressed precisely. Disagreements between operators are preserved rather than resolved automatically, and conflicting authority actions result in refusal rather than compromise unless an explicit coordination rule permits resolution.

By treating authority scope and roles as first-class, governed constructs, the system remains correct under both single-operator and multi-operator deployments. The presence of multiple operators does not introduce ambiguity, dilution, or drift, because authority semantics are enforced structurally rather than inferred from interaction patterns. This completes the authority model required to reason about correctness under accumulation and sets the stage for examining determinism, provenance, and replay across complex governance configurations.

10. Provenance, Determinism, and Replay Across Authority Configurations

The authority model is only as strong as its ability to explain and validate past decisions. For a system that accumulates state indefinitely and supports correction of authoritative error, it must be possible to reconstruct not only what decisions were made, but under which authority configuration they were justified. Provenance, determinism, and replay jointly provide this capability.

Provenance in this architecture is an append-only record of committed state transitions, inputs, and authority configurations. Each committed action records the authoritative basis under which it was permitted, including the relevant Tier C state, the scope of authority exercised, and the Substrate version enforcing the rules at the time. In this architecture, provenance records not only dependency relationships but the authority conditions under which actions were admissible.

Deterministic execution is a necessary precondition for meaningful provenance. Given the same inputs, governed mutable state, authority configuration, and Substrate version, the system must produce the same outcome. This determinism ensures that provenance entries are not narrative reconstructions but verifiable consequences of system law. Without determinism, provenance would devolve into explanation rather than evidence.

Replay leverages determinism to make authority inspectable and correctable. By re-executing a past request under its original authority configuration, the system can demonstrate why a decision was permitted at the time. By re-executing that same request under a different authority configuration, the system can determine whether the decision would still be admissible. This capability is central to auditing, debugging, and governance review.

Replay operates over fixed configurations. It does not reinterpret historical decisions under current rules unless explicitly requested for comparison. This distinction preserves historical truth while enabling counterfactual analysis. Reversion changes which authority configurations are active for future execution while leaving all prior provenance entries intact.

In multi-operator contexts, provenance and replay remain well-defined because authority configurations are explicit and versioned. Each action is attributable to a specific scope and role configuration, and replay faithfully reconstructs that context. Disagreements among operators or corrections to authority result in new provenance entries that supersede prior authority going forward. The record remains unambiguous.

Provenance, determinism, and replay ensure that authority within the system remains explicit. Decisions can be traced to an explicit epistemic basis, corrections can be justified without erasure, and claims of correctness can be validated mechanically. The system supports accumulation without drift, evolution without implicit change, and long-horizon operation without loss of accountability.

11. Limitations, Non-Goals, and Scope

The architecture presented in this paper is intentionally narrow in what it attempts to guarantee. Its goal is not to produce correct answers, aligned behavior, or optimal decisions, but to ensure that any authority exercised by the system is explicit, governed, inspectable, and reversible under accumulation. Several important limitations and non-goals follow directly from this focus.

L1. The system does not attempt to determine truth. Authoritative state reflects explicit governance decisions, not objective correctness. The architecture ensures that the basis for decisions is known and reviewable, but it does not adjudicate whether that basis is factually accurate. Incorrect or incomplete information may be promoted into authoritative state, and the system remains correct so long as such promotion is explicit, attributable, and reversible.

L2. The system does not prevent human error or poor judgment. Operators may approve incorrect promotions, define inadequate scopes, or make governance decisions that later require correction. The architecture is designed to make such errors visible and correctable, not to eliminate them. Safety arises from bounded authority and explicit correction paths, not from assumptions about operator infallibility.

L3. The system does not automate governance or social decision-making. While it may assist by generating analyses, organizing information, or assembling promotion proposals, it does not resolve disagreement, infer consensus, or substitute algorithmic judgment for human authority. Organizational processes, trust relationships, and accountability structures remain external to the system and must be expressed explicitly if they are to be enforced [9].

L4. The guarantees described here are not achievable through access control, policy enforcement, provenance tracking, or procedural governance alone. Access control constrains who may act but does not constrain how accumulated artifacts acquire epistemic authority under reuse. Policy systems and procedural rules remain mutable and therefore permit authority semantics to change implicitly over time. Provenance captures dependency and lineage but does not, by itself, determine admissibility or epistemic status. These mechanisms are valuable and often necessary, but without immutable enforcement of authority transitions they are insufficient to prevent implicit authority drift under accumulation.

L5. Any system in which authority semantics reside in mutable policy, process, or interpretation admits implicit authority drift under accumulation by construction. Preventing such drift requires that authority be represented as an explicit system property whose transitions are governed and enforced by immutable law.

L6. The system does not optimize for performance, adaptability, or learning. It imposes structural constraints that may limit convenience or throughput in favor of correctness under accumulation. Designs that prioritize rapid adaptation, implicit learning, or continuous self-modification fall outside the scope of this work.

L7. This paper does not specify concrete authorization policies, authentication mechanisms, device trust models, or interaction modalities. The architecture permits such constraints to be expressed as part of authority configurations, but it does not enumerate or standardize them. Doing so would prematurely bind the system to particular operational contexts. More detailed enforcement schemes are deferred to future work.

Within these bounds, this paper defines a set of necessary conditions and a formal system structure that make implicit authority drift structurally impossible. Systems that fall outside these constraints may be effective in other respects, but they cannot provide the same guarantees of long-horizon correctness and inspectability under unbounded accumulation. Any system that accumulates and reuses derived artifacts over long horizons must enforce equivalent constraints on authority representation and transition in order to prevent implicit authority drift.

12. Discussion and Conclusion

Correctness in long-horizon AI systems is reframed here as a property of explicit authority, admissibility, and governance rather than of output quality, alignment, or optimization. It addresses a specific structural failure mode that emerges under accumulation: the implicit drift of epistemic authority through reuse, dependency, and convenience. By treating authority as a first-class, governed concept enforced by immutable system law, the architecture makes this failure mode structurally impossible rather than procedurally discouraged.

A central implication of this design is that correctness is framed in terms of permission and admissibility rather than outcome quality. The system remains correct even when it produces incorrect analyses, incomplete research, or misguided proposals, provided that such artifacts are prevented from acquiring authority without explicit governance. This reframing distinguishes the architecture from approaches that attempt to ensure safety by improving model behavior, alignment, or confidence estimation. Here, safety is achieved by constraining what information is permitted to ground decisions, not by attempting to ensure that all information is reliable.

The architecture also clarifies the role of determinism, provenance, and replay in long-horizon governance. Provenance is not treated as merely an audit trail, but as a record of epistemic admissibility under specific authority configurations. Deterministic replay enables past decisions to be inspected, contested, and compared under alternative authority assumptions without reinterpretation or erasure. Together, these mechanisms support correction of authoritative error while preserving historical truth, allowing the system to evolve without accumulating uninspectable or irreversible commitments.

While this paper focuses on a minimal core, the broader design space includes additional constraints and mechanisms that were deliberately excluded to preserve conceptual clarity. These include explicit modeling of authority scopes and roles, non-compressible authority assignment, consent-gated execution envelopes, and structural accommodations for human governance limits such as operator fatigue and consolidation. These elements do not alter the core argument, but they demonstrate that the architecture can be extended to support complex, multi-operator deployments without relaxing its fundamental invariants. Such extensions are more appropriately treated as instantiations of the core model rather than prerequisites for its correctness.

The resulting system is therefore best understood as a foundation rather than a complete solution. It provides a formal substrate in which long-horizon AI systems can accumulate context, perform research, and assist human decision-making without silently acquiring authority. Domains characterized by high cost of failure, long operational timelines, and the need for explicit accountability may benefit from such a foundation, even when it imposes constraints on adaptability or convenience. Other architectural paradigms may be better suited to applications that prioritize rapid optimization or autonomous adaptation [2].

In closing, this work argues that authority drift is not an emergent property to be mitigated, but a structural failure to be prevented. By separating immutable enforcement from governed state, making authority transitions explicit and reversible, and enforcing determinism over accumulation, the architecture enables systems to evolve without implicit change in what they are permitted to rely upon. This shift in framing moves the problem of long-horizon AI safety from behavior to structure, and from trust to enforceable system law. A full formal specification and proof of the architecture's invariants are deferred to a longer companion work, in which the claims advanced here are made precise and verified within an explicit formal system using TLA+.

Appendix A: Definitions

Accumulation
The reuse of state or artifacts across decisions such that prior outputs may influence subsequent behavior. In this work, accumulation is treated as a structural condition independent of time, scale, or duration.

Authority
The system-enforced admissibility of information or state to ground decisions or authorize actions. Authority is not inferred from confidence, frequency of use, coherence, or dependency, but is assigned explicitly through governed transitions enforced by the Substrate.

Authority Configuration
The complete, versioned context defining which authority scopes, roles, and Tier C state are active for the purpose of validating a request. Authority configurations bound replay, admissibility, and reversibility; changes to authority configuration constitute governed transitions.

Authority Scope
The bounded domain within which an operator or role may exercise authority. Scope is explicit, enforced by the Substrate, and non-aggregative; authority does not emerge from overlap, consensus, or repeated reliance unless explicitly governed.

Deterministic Replay
The ability to re-execute a prior request under an identical Substrate version, authority configuration, and governed mutable state and obtain the same outcome. Replay serves as evidence of admissibility and justification, not as explanation or approximation.

Epistemic Authority (Epistemic Status)
The designation that information or state is permitted to serve as justification for system actions or decisions under Substrate-enforced rules. In this work, epistemic status does not refer to belief, confidence, truth, or knowledge, but solely to admissibility for grounding action.

Governance
The formal, Substrate-enforced process by which authority transitions are proposed, validated, approved, or rejected. Governance is structural rather than procedural: it is implemented as invariant system rules, not as informal process, policy documents, or social convention.

Long-Horizon
Describes systems in which accumulation of state and reuse across decisions is unavoidable. The term refers to structural inevitability rather than temporal length or operational lifespan.

Promotion
A discrete, governed state transition that assigns epistemic authority to specific state within the Strata, resulting in its inclusion in authoritative Tier C. Promotion is explicit, attributable, and enforced by the Substrate; gradual, implicit, or usage-based elevation of authority is prohibited.

Provenance
An append-only record of committed state transitions, inputs, and authority configurations. In this architecture, provenance captures not only dependency relationships but the epistemic status under which decisions were admissible at the time they were made.

Refusal
A correctness-preserving pipeline outcome in which a request is not executed because its satisfaction would violate Substrate-enforced authority semantics or invariants. Refusal is not an error condition, degraded behavior, or exceptional state, but a required outcome when admissibility conditions are not met.

Strata
The totality of mutable system state governed by the Substrate. Strata includes all accumulated artifacts, configurations, and data that may evolve over time through Substrate-validated transitions. The Strata contains no logic capable of reinterpreting, weakening, or modifying Substrate-enforced rules.

Substrate
The immutable enforcement machinery of the system. The Substrate defines authority semantics, governance constraints, validation rules, and admissibility conditions for state transitions. It is version-static at runtime and cannot be modified by any process operating within the system. The Substrate does not learn, adapt, or accumulate state; its sole function is to enforce invariants deterministically.

Appendix B: Illustrative Failure Mode Under Accumulation

This appendix provides a minimal illustrative example of the failure mode discussed in Sections 1 and 2. It is not an implementation sketch, usage recommendation, or empirical claim. Its purpose is solely to clarify how implicit authority drift can arise under accumulation in the absence of enforced authority semantics, even when all individual steps are locally correct and well intentioned.

Consider a system that processes repeated requests and is permitted to retain derived artifacts across executions. The system treats all retained artifacts as advisory by convention, but does not enforce any structural distinction between advisory context and epistemic basis. Provenance is recorded, and all artifacts retain traceable dependency histories.

Initially, the system processes a request and produces a derived artifact A(1). A(1) is labeled advisory. It summarizes inputs, aggregates information, or proposes a structured interpretation. No authority is granted, and no rules are violated.

On a subsequent request, the system consults A(1) as context. Doing so is locally rational: A(1) reduces repeated work and improves coherence. The system produces a new artifact A(2) that depends on A(1). Provenance records this dependency accurately. A(2) is also labeled advisory.

Over time, the system accumulates artifacts A(1)…A(n), each derived from prior artifacts and original inputs. None are explicitly promoted. No artifact is ever treated as authoritative in isolation. However, new decisions increasingly depend on the consistency, structure, and conclusions of this artifact set rather than on the original epistemic basis.

At this point, the system reaches a state where a new decision D depends on A(n), which depends on A(n-1), and so on. Provenance can show the full chain of derivation. What it cannot answer is a different question: which elements of this chain were permitted to ground D at the time it was made.

No discrete transition marked the moment at which advisory context became epistemic basis. No rule was violated. Each reuse was permitted. Each dependency was recorded. Yet the system's behavior is now grounded in accumulated derived artifacts whose epistemic status was never explicitly authorized.

If an operator attempts to correct the system, there is no clear boundary to revert, even with full access to provenance and historical artifacts. Removing A(n) does not restore a known-good state, because A(n-1) already incorporates its assumptions. Reverting further collapses large portions of accumulated context without a principled stopping point. The system cannot replay D under a prior authority configuration because no such configuration was ever made explicit.

This illustrates the core claim of the paper: implicit authority drift does not arise from error, malice, or hidden behavior, but from accumulation in the absence of enforced authority semantics. Provenance alone is insufficient when it records dependency without enforced epistemic admissibility. Without explicit, governed transitions of authority enforced by immutable system rules, reuse alone is sufficient to undermine long-horizon correctness.

Appendix C: An Abstract Model of Authority Drift

This appendix provides a minimal abstract model intended to clarify the failure mode described in Sections 1 through 3 and to justify, at a structural level, the architectural constraints proposed in Sections 4 through 10. The purpose of this appendix is not to present a complete formal specification or a machine-checked proof. Rather, it defines authority drift as a precise system property and explains why explicit authority-granting transitions are required to prevent it in accumulating systems.

The model intentionally abstracts away implementation details, learning algorithms, internal representations, and optimization concerns. It focuses only on how system state evolves over time, how actions are judged admissible, and how accumulated artifacts come to influence those judgments.

C.1 Accumulating Systems

We define an accumulating system as a system that operates over persistent state and processes a sequence of decisions over time.

Such a system has the following characteristics:

1. The system has a notion of state, denoted S, which persists across decisions and executions.
2. The system executes actions that transition the system from one state to a successor state.
3. The system has an admissibility predicate, written as Adm(S, a), which determines whether a given action a is permitted to execute in state S. Adm(S, a) returns either true or false.
4. State persistence holds: artifacts present in a state remain present in successor states unless explicitly removed by a transition.
5. Artifact reuse holds: artifacts present in state S may be consulted, directly or indirectly, when constructing actions or when evaluating whether future actions are admissible.

No restriction is placed, at this level, on how admissibility is computed or which parts of state it may consult. In particular, Adm(S, a) may depend on any portion of accumulated state.

This definition is intentionally broad. It captures a wide range of long-running systems, including autonomous agents, decision-support systems, and interactive AI systems that retain and reuse internal artifacts across executions.

C.2 Operational Authority and Authority Drift

To reason about authority drift without presupposing the architecture proposed in this paper, we introduce an operational notion of authority.

For a given state S, let D(S) denote the subset of state elements that are actually consulted by Adm(S, a) when determining admissibility for actions in that state. Elements of D(S) are said to be operationally authoritative at state S.

This notion of authority is descriptive rather than normative. It does not assert that such elements ought to be authoritative. It captures only which elements function as authoritative in practice by grounding admissibility decisions.

Using this notion, we define authority drift as follows.

An accumulating system exhibits authority drift if there exists a sequence of states S(0), S(1), …, S(n) and an element x such that:

- x is not consulted by the admissibility predicate in the initial state S(0),
- x is consulted by the admissibility predicate in a later state S(n), and
- no discrete transition along the sequence explicitly grants x the right to influence admissibility.

Intuitively, authority drift occurs when an artifact comes to ground admissibility decisions through accumulation and reuse rather than through an explicit authorization step.

This definition is independent of any particular architectural remedy. It characterizes a failure mode in terms of observable system behavior.

C.3 Reuse-Sensitive Admissibility

Not all accumulating systems necessarily exhibit authority drift. A degenerate system could accumulate state while never consulting it for admissibility decisions. The failure mode of interest arises when admissibility is reuse-sensitive.

An accumulating system is said to have reuse-sensitive admissibility if there exists at least one execution in which:

- new artifacts are added to system state over time, and
- the set of state elements consulted by the admissibility predicate is permitted to expand as a function of accumulated artifacts.

This condition captures the common case in long-horizon systems where prior analyses, summaries, plans, heuristics, or learned constraints increasingly influence what actions are considered permissible.

C.4 Conditional Authority Drift Lemma

We can now state the central observation supported by this abstract model.

In an accumulating system with reuse-sensitive admissibility, authority drift is structurally permitted unless the system enforces explicit authority-granting transitions that constrain when accumulated artifacts may influence admissibility.

The argument is straightforward. Because admissibility is reuse-sensitive, there exists an execution in which newly accumulated artifacts are permitted to affect admissibility decisions. In the absence of enforced constraints on when such influence is allowed, there exists an execution in which an artifact not previously consulted by the admissibility predicate becomes consulted solely through reuse and dependency. By the definition above, this constitutes authority drift.

The claim is conditional rather than universal. It does not assert that drift occurs along all executions, but that drift is structurally permitted and therefore unavoidable as a class property unless explicitly prevented.

C.5 Architectural Constraint: Promotion-Only Authority

The architecture proposed in this paper prevents authority drift by construction through a restriction on how admissibility may depend on system state.

Under the proposed architecture, system state is partitioned into non-authoritative artifacts and authoritative epistemic basis. The admissibility predicate may depend only on authoritative state. Transitions that add elements to authoritative state are discrete, governed, and explicitly designated as authority-granting promotions.

Under this constraint:

- the set of state elements consulted by the admissibility predicate is fixed relative to the current authority configuration, and
- accumulation of non-authoritative artifacts cannot expand admissibility dependence.

As a result, authority drift as defined above cannot occur.

C.6 Structural Implication

The preceding definitions and lemma support the following structural claim.

Any accumulating system that both permits reuse-sensitive admissibility and prevents authority drift must enforce constraints equivalent in effect to the promotion-only authority invariant described above. In particular, such a system must include:

- a distinguished subset of state whose elements are permitted to ground admissibility decisions,
- explicit transitions that grant elements membership in that subset, and
- enforcement logic that prevents admissibility from depending on non-authoritative artifacts.

This claim is not an implementation mandate. Different systems may realize these constraints through different mechanisms. The claim is that some structurally equivalent separation must exist to block the drift phenomenon defined in this appendix.

C.7 Scope and Limitations

This appendix does not attempt to provide a complete taxonomy of admissibility semantics, to prove minimality across all conceivable system models, or to mechanize the argument in a proof assistant.

Its purpose is narrower: to define authority drift precisely, to explain why accumulation combined with reuse-sensitive admissibility permits it, and to justify why the architectural constraints proposed in this paper are structural requirements rather than stylistic choices.

A full formalization of this model, including a concrete transition system and machine-checked invariants, is a natural direction for future work but is not required to understand or apply the architectural principles described here.

References

[1] Alpern, B., and Lamport, L.
Recognizing safety and liveness.
Distributed Computing, 2(3):117-126, 1987.

[2] Garlan, D., and Shaw, M.
An introduction to software architecture.
In *Advances in Software Engineering and Knowledge Engineering*, vol. 1, pp. 1-39. World Scientific, 1993.

[3] Gray, J., and Reuter, A.
Transaction Processing: Concepts and Techniques.
Morgan Kaufmann Publishers, San Mateo, CA, 1992.

[4] Hoare, C. A. R.
An axiomatic basis for computer programming.
Communications of the ACM, vol. 12, no. 10, pp. 576-583, 1969.

[5] Lamport, L.
The temporal logic of actions.
ACM Transactions on Programming Languages and Systems, 16(3):872-923, 1994.

[6] Lamport, L.
Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.
Addison-Wesley, Boston, MA, 2002.

[7] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., and Schwarz, P.
ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks.
ACM Transactions on Database Systems, 17(1):94-162, 1992.

[8] Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., and Van den Bussche, J.
The PROV data model.
W3C Recommendation, 2013.

[9] Saltzer, J. H., and Schroeder, M. D.
The protection of information in computer systems.
Proceedings of the IEEE, 63(9):1278-1308, 1975.

[10] Schneider, F. B.
Enforceable security policies.
ACM Transactions on Information and System Security, 3(1):30-50, 2000.